Task3

Loshmanov Egor

Описание вычислительного узла: Вычислительный узел оснащён двумя процессорами Intel Xeon Gold 6248 (40 физических ядер, 80 логических потоков, частота до 3.9 ГГц, кэш L3 — 55 МБ, архитектура x86_64, 2 NUMA-узла).
Наименование сервера: ProLiant XL270d Gen10
Кол-во NUMA node: 2 nodes (0-1)
node 0 size: 385636 MB
node 1 size: 387008 MB
OS: Ubuntu 22.04.5 LTS

Таблица времени выполнения и ускорение программы при различном числе потоков и реализации

n (размер матрицы)	Вариант	Количество потоков												
		1	2		4		8		16		20		40	
		T1	T2	S2	T4	S4	T8	S8	T16	S16	T20	S20	T40	S40
2000	Вариант 1	35,963992	16,522561	1,954354	7,093491	4,552191	2,73937	11,78774	2,245277	14,381713	2,139975	15,089394	5,58643	5,780243
	Вариант 2	35,605429	16,312142	1,979564	7,573641	4,263593	3,02856	10,66216	2,40011	13,453937	2,616897	12,339395	5,92713	5,447988

где:

- n — размер векторов и матрицы системы;
- Tp - время выполнения программы при p потоков;
- Sp - ускорение при p потоков.

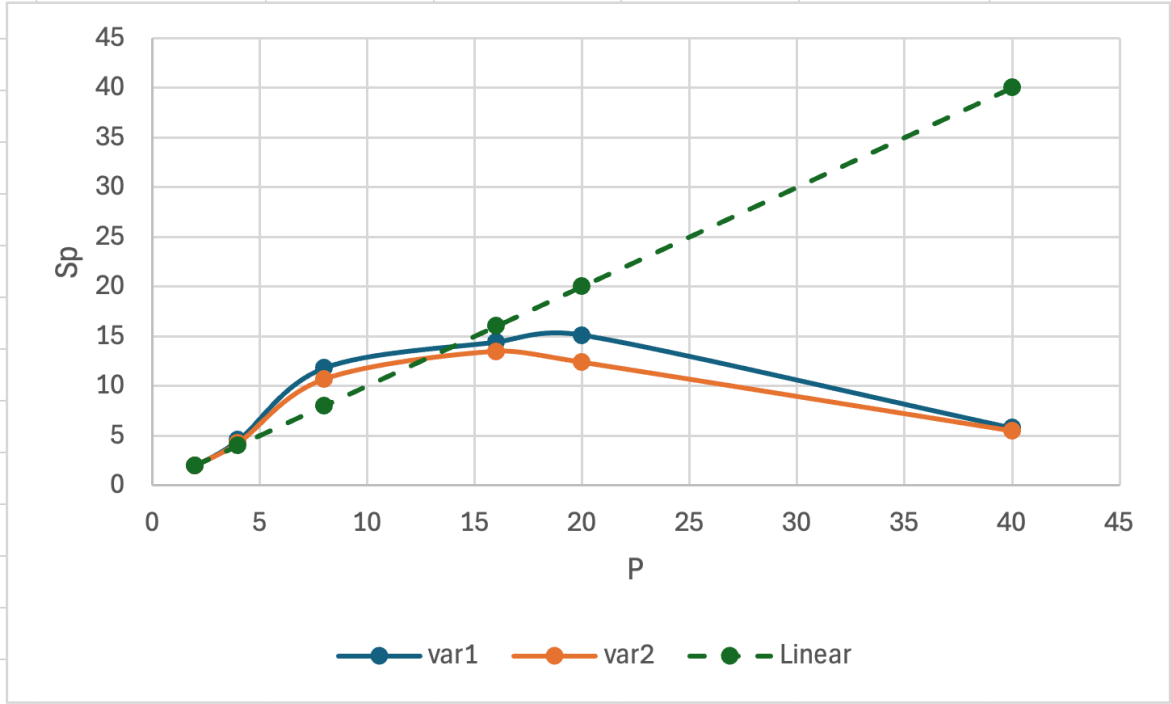
Формула для вычисления ускорения

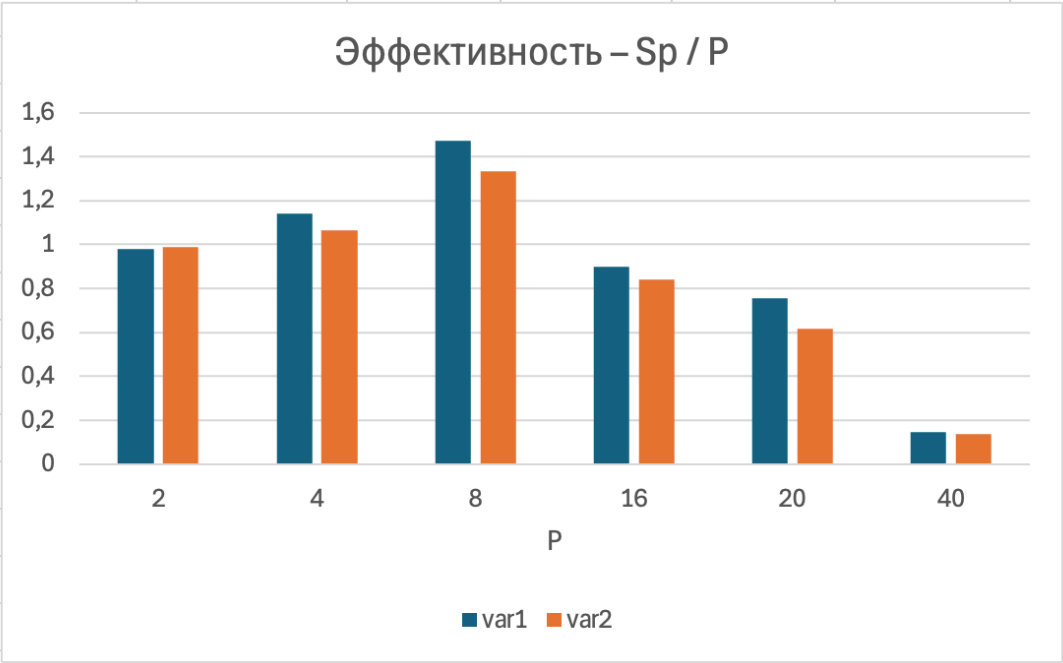
$$S_p = \frac{T_1}{T_p}$$

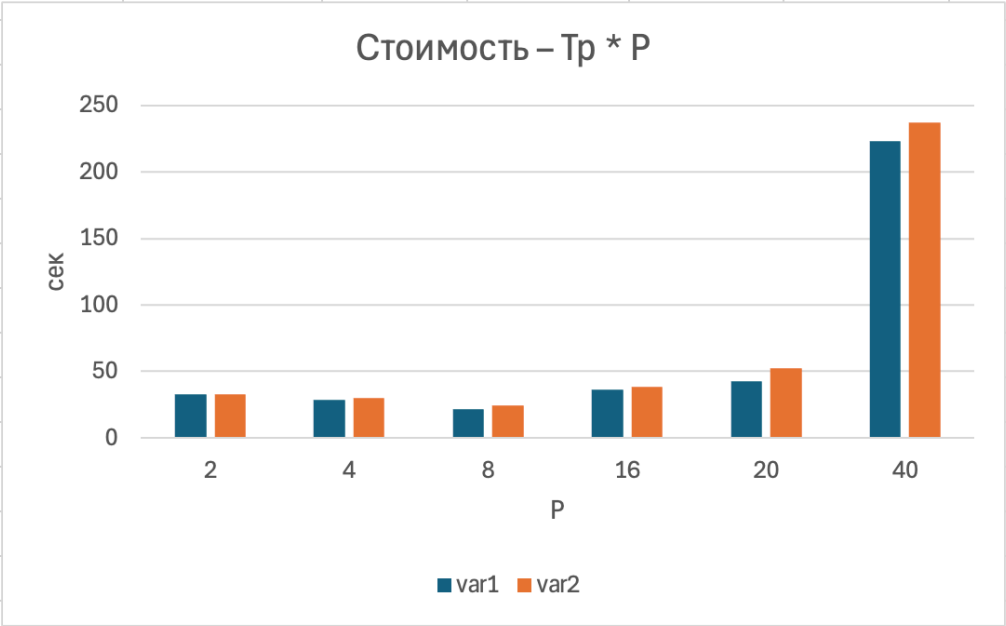
Вариант 1 — на каждую параллельную операцию внутри итерации создаётся своя параллельная секция

Вариант 2 — потоки создаются один раз, синхронизируются барьерами внутри

График зависимости ускорения Sp от числа потоков p







static — оптимальный для данной задачи (t=3.52, S=9.17).

Почему:
- Матрично-векторное умножение - равномерная нагрузка: каждая строка матрицы требует одинакового объёма вычислений
- static делит строки поровну без накладных расходов на планирование -> минимальные потери
- dynamic медленнее (t=4.76) - каждый поток запрашивает следующий chunk у планировщика -> лишняя синхронизация при равномерной нагрузке
- guided средний (t=3.84) - убывающий chunk частично снижает накладные расходы, но всё равно хуже static

Вывод: для задач с равномерной нагрузкой на итерацию (каким является dgemv внутри итерационного метода) оптимален schedule(static). dynamic и guided целесообразны только при неравномерной нагрузке.

По графикам первый вариант стабильно показывает более высокое ускорение и эффективность при всех значениях p, особенно заметно при 8–20 потоках. Накладные расходы второго варианта выше из-за барьеров синхронизации внутри единой параллельной секции, тогда как первый эффективнее использует неявные барьеры OpenMP. Таким образом, для данной задачи Вариант 1 предпочтительнее, однако при очень большом числе итераций преимущество Варианта 2 (однократное создание потоков) может стать ощутимым.